Instructions: https://canvas.eee.uci.edu/courses/80356/pages/projects-for-cs178-winter
Dataset: https://archive.ics.uci.edu/dataset/186/wine+quality
GitHub: https://github.com/AlistairKeiller/cs178Final/

Report Checklist

- [X] summary (2-4 sentences)
- [X] data description
  - [X] description
  - [X] 1 figure
  - [N] 1 paper
- [ ] classifiers
  - [X] KNN
    - [X] 1-2 sentence per classifier
    - [X] hyperparameters tuned
    - [X] libraries used
  - [X] NN
    - [X] 1-2 sentence per classifier
    - [X] hyperparameters tuned
    - [X] libraries used
  - [ ] random forest
    - [ ] 1-2 sentence per classifier
    - [ ] hyperparameters tuned
    - [ ] libraries used
  - [X] Logistic
    - [x] 1-2 sentence per classifier
    - [x] hyperparameters tuned
    - [x] libraries used
- [ ] experimental setup
  - [X] KNN
    - [x] metrics
    - [x] how hyperparameters were selected
  - [X] NN
    - [X] metrics
    - [X] how hyperparameters were selected
  - [ ] random forest
    - [ ] metrics
    - [ ] how hyperparameters were selected
  - [X] Logistic
    - [X] metrics
    - [X] how hyperparameters were selected
- [ ] experimental results
  - [X] interpretation of results
  - [X] table/figure
    - [x] KNN

- [x] NN
- [x] random forest
- [ ] Logistic
- [x] comparison of performance

- [x] insights
  - [x] summary of what we learned
  - [x] any surprising results
  - [x] “error” analysis
    - [x] weaknesses of the classifiers & comparison with each other and humans
    - [x] strengths of the classifiers
    - [x] (optional) speculation on performance on real world data

- [ ] contributions
  - [ ] 1-2 sentence for what each person did
    - [x] Alistair
    - [x] Cindy
    - [ ] Teck
    - [ ] Tanisha

- [x] link to github

An Analysis of Machine Learning Approaches on the UCI Wine Dataset

Teck Ho, Alistair Keiller, Tanisha Singh, Cindy Zhang

I. Summary

In this paper, we compare the efficacy of kNN classifiers, logistic classifiers, neural networks, and random forests when predicting wine quality based on the wine's physicochemical properties. We then analyze the training curves of each model, and how their parameters interact with training data of various different sizes. Finally, we create an ensemble model we would use in production for optimal accuracy.

II. Data Description

The dataset used in this paper is Wine Quality from the UCI Machine Learning Repository. This dataset includes 11 features across 4898 data points collected from red and white vinho verde wine samples. Our goal is to predict wine quality, as tasters rated it between 0 and 10.

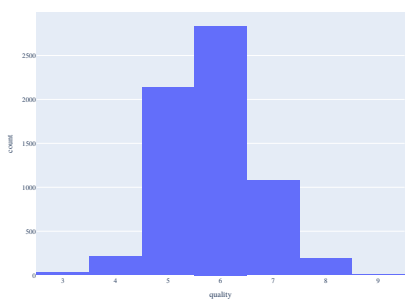

Fig. 1. Distribution of quality as rated by tasters between 0 and 10

Based on Fig. 1, we observed the quality data to be approximately normal, with a mean of 5.8 and a standard deviation of 0.87. As a result, there are no instances of quality 0 through 2 or 10, and there are very few instances of quality 3 and 9. Based on this analysis, we can remove the bins 0-2 and 10 from our models output range, since we have no training data on those qualities. We then computed the correlation matrix.

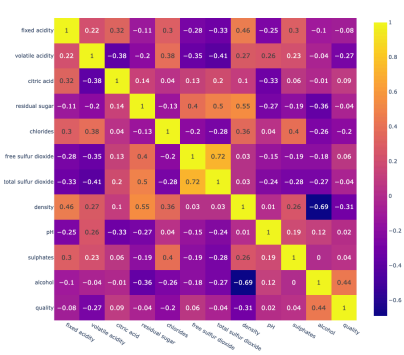

Fig. 2. Correlation between features

Using Fig. 2, we determined that alcohol content was the feature most correlated with quality, with a correlation of 0.44. We then plotted alcohol against quality and color.

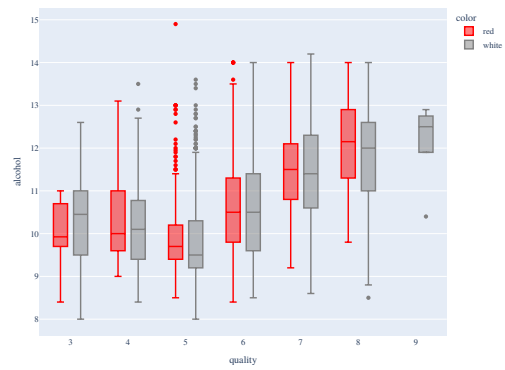

Fig. 3. Alcohol vs color and quality

Reviewing Fig. 3, we determined that low alcohol quantity is correlated with mediocre wine, low to medium alcohol quantity is correlated with low quality wine, and high alcohol quality is correlated with high quality wine.

In order to interpret models, we ideally want two features that are highly correlated with quality but not with each other. While density is the feature with the second highest correlation to alcohol, it is also highly correlated with alcohol. Therefore, instead we will use volatile acidity, with correlation -0.27.

III. Classifiers

A. kNN

kNN classifiers make predictions on points based on majority vote from their k nearest neighbors. For our implementation, we used `KNeighborsClassifer` from `scikit-learn`. The hyperparameters we tuned were the set of features used and the parameter k. To choose a set of features, we used the `itertools` library's `combinations` function to loop through all combinations.

B. Logistic

Logistic regression classifiers make their predictions based on the output of feeding input data through a logistic function. When using binary data, this is traditionally any continuous function, such as a sigmoid. With multiclass data, functions like softmax are more commonly applied. The classifier trains to minimize its loss, calculated as cross by constantly modifying its weights according to the inputs. To do so, we primarily tuned its solver, type of regularization, and extend of regular-

ization, C. We used `sklearn`'s `LogisticRegression` classifier.

C. Neural Network

Neural network classifiers make their predictions by taking an input vector, feeding it through layers by using matrix multiplication, addition, and activation functions, and then then final layer is the output layer, which has the resulting output. In our case, we have 7 output nodes corresponding to each quality class (3-9). We implemented a Neural Net in PyTorch, which provides better training speed, more information about the training process, and the ability to customize the models. For example, we use dropout layers in training as a method to prevent overfitting.

D. random forest

random forest classifiers generate predictions by combining multiple decision trees. To implement the random forest algorithm, using `RandomForestClassifier` from `sklearn.ensemble` makes the model more robust. Increasing the number of trees via `n_estimators` typically improves the accuracy score of the data. `max_depth` allows deeper trees to expand in the complexity as they produce more splits.

IV. Experimental Setup

We split our data into 20% testing, 20% validation, and 60% training. Our goal is to predict the wine quality, so we split each quality value into its own class. In order to tune or models, we plotted validation accuracy for each set of hyperparameters in a graph. We will use more detailed metrics in the results, but we decided to tune off of accuracy since it is the single metric most representative of a models usefulness.

We also find later that the model overfit to the disproportionate amount of data in classes 5 through 7. This helps improve accuracy on testing, but would result in the model very rarely giving high or low scores. Stratifying the data is extremely difficult, since there are too few points in class 9 to cut down the data to equal class sizes, and duplicating points causes issues with models like KNN overfitting. In future work, we propose stratifying data by weighting it according to class size.

A. kNN

To get the best kNN classifier, we trained models against different sets of features and against different values of k.

TABLE I
Each feature name and its corresponding index.

Index	Feature Name
0	fixed_acidity
1	volatile_acidity
2	citric_acid
3	residual_sugar
4	chlorides
5	free_sulfur_dioxide
6	total_sulfur_dioxide
7	density
8	pH
9	sulphates
10	alcohol
11	color

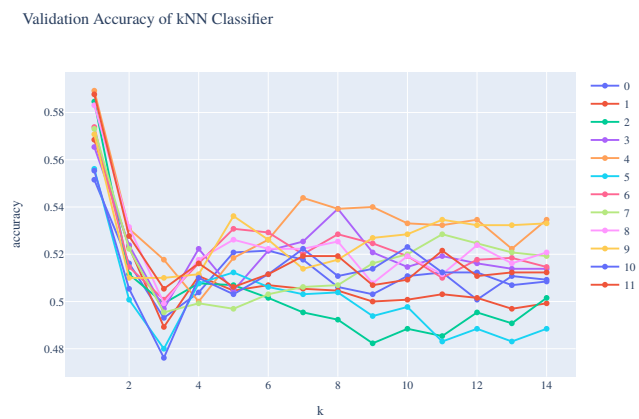

Fig. 4. Validation accuracy of kNN models when removing 1 feature over different values for k.

As a sample of what the training results were, Fig. 4 shows that removing certain features individually like feature 5 results in higher accuracy. Due to how many combinations of sets of features there are and the low readability of the graphs it results in, we are omitting a visualization of the accuracy curves for all the hyperparameters tuned. The hyperparameters that resulted in the best validation accuracy at 60.4% used features [0, 1, 2, 3, 4, 8, 9, 10] and $k = 1$.

In addition, as k increases, the accuracy drops significantly because we haven't removed duplicates in the data. Datapoints from a different class that are close to a cluster of duplicates will be classified the same as the duplicates because the range of neighbors won't be able to extend past the duplicates.

B. Logistic

Logistic regression involves fewer hyperparameters to tune than other models. The solver and penalty types, respectively, made the most difference on the training and validation accuracy rates.

Certain solvers cannot work with every penalty type, so the learning curves below contain only valid combinations of the two.

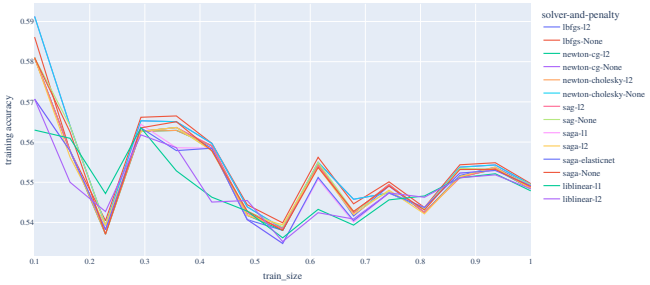

Fig. 5. Training accuracy across different solvers and penalties

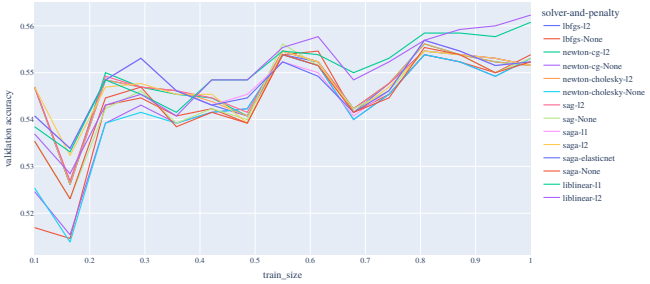

Fig. 6. Validation accuracy across different solvers and penalties

Not shown in this graph, the C parameter delineating the extent of the regularization is tuned for every model.

Note that the liblinear solver is suited solely for binary data, so it uses the One-vs-Rest approach to identifying multiple classes. This involves splitting a multi-class dataset into multiple binary classifiers, with one per class in the original dataset. This solver seems to have been the most effective with the correct parameters, but it is important to note that it can become slow and inefficient in large datasets with lots of classes, since it will train one model per class.

`LogisticRegression` seems to increase in validation accuracy for every increase in training dataset size, indicating it can improve its ability to fit gradually, as opposed to other models such as neural networks that pick up more quickly but stagnate in growth. However, its peak accuracy seems to be smaller than those of more modern alternatives, likely because it cannot capture more complex, nonlinear relationships.

The magnitude of the C parameter that yields the highest accuracy also demonstrates how well a model responds to regularization. Recall that C is inversely proportional to regularization. Solvers such as lbfgs, newton-cg, newton-cholesky, and sag benefit from a higher value of C, 100 for l2 regularization, and therefore less regularization. Note that most of these models tend to perform better on larger datasets, so it is possible they need to be able to overfit on smaller datasets.

The solvers that are more suited to small datasets, saga and lib-linear, both performed the best on our dataset but also had more middling ideal C values such as 1 and 2 for varying penalty types. This indicate they do not need to overfit as much since they are well-suited to learning more quickly on small datasets.

Ultimately, narrowing down the ideal solver and penalty combination yield a training accuracy of 54.79%, validation accuracy of 56.08%, and testing accuracy of 53.08%. The ideal classifier was liblinear with penalty type l1 and C of 1

C. Neural Network

Well chosen hyperparameters are paramount to a accurate Neural Network. The first hyper parameter we tune is the hidden layer sizes. Given that the dataset is small, we can assume that we will want a relatively small network to combat overfitting. An initial guess is that the network should be 16, 32, 64, or 128 nodes, and either 1 or 2 hidden layers.

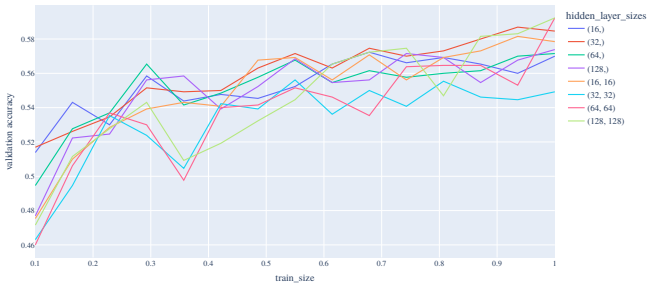

Fig. 7. Validation accuracy across hidden layer sizes

Fig. 7 shows the validation accuracy for each set of hidden_layer_sizes. For small train_size, the smaller models perform much better, since they overfit to the training data less than the large models. However, as train_size grows, the larger models overfit less, and bridge the gap with the smaller models. With all of our training data, the largest models with size $(128, 128)$ and $(64, 64)$ are the best. This is surprising, since that is relatively large for a small dataset. Moving forward, we will use hidden_layer_sizes $(128, 128)$ instead of $(64, 64)$ since regularization and dropout aim to further reduce overfitting, they have a larger potential to improver accuracy on the larger model.

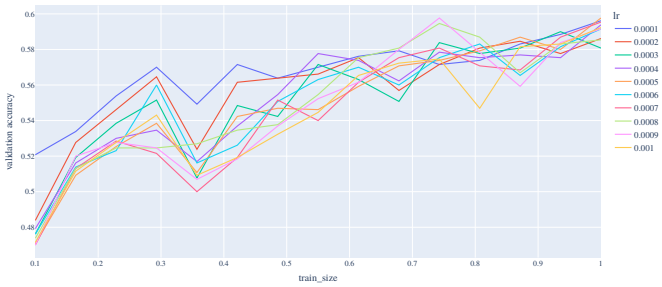

Fig. 8. Validation accuracy across learning rates

From Fig. 8, we find that a small learning rate is helpful with small train_size, in order to prevent overfitting to the data. However, with train_size=1, the differences are negligible, so we will just go with the highest validation accuracy of 0.005.

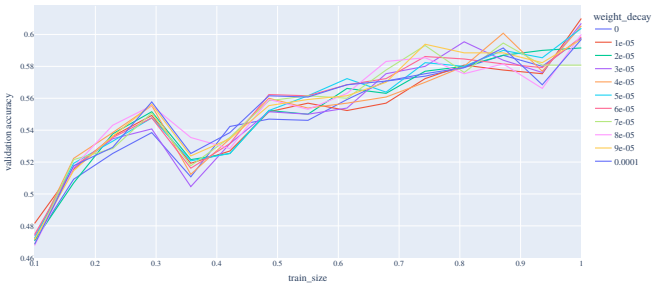

Fig. 9. Validation accuracy across weight decay

Fig. 9 shows the validation accuracy for each weight decay. The weight decay is a type of L2 regularization, which punishes large weights and biases, therefore reducing model complexity and overfitting. The weight decay with the highest validation score is $1 \cdot 10^{-5}$.

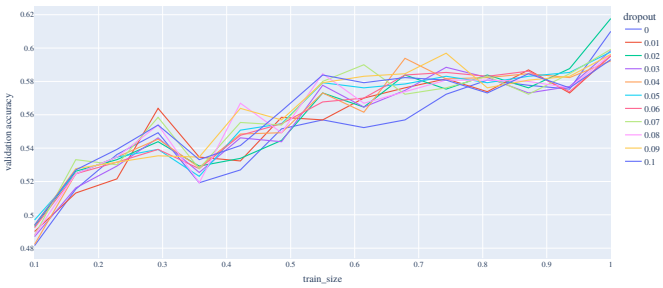

Fig. 10. Validation accuracy across dropouts

Fig. 10 shows the validation accuracy for each dropout. A dropout layer is a layer that will randomly drop neurons while training. This forces the network to become more robust, and reduces overfitting. Small values of dropout are helpful, but too large a value deceases our validation accuracy. The dropout with the highest validation score is 0.02.

Combing our results, we have a model with 2 hidden layers of size 128, a leanring rate of 0.0005, a weight_decay of 0.00001, and a dropout of 0.02. This results in a validation accuracy of 0.6177. But if we tune all the values at once we can do better. Based on all our findings, I created an optuna hyperparamater optimization run. Since our model benefited so much from being larger, I gave it a much larger model size, with either 128, 256, or 512 nodes per layer and 1-4 hidden layers. I gave it a learning rate range of $(10^{-4}, 10^{-2})$, a dropout range of $(0, 0.3)$, a weight decay range of $(10^{-6}, 10^{-2})$, and a batch_size range options of 128, 256, 512, 1024, or 2048. The optimal values were a 2 layer network with 512 node layer sizes, a learning rate of 0.002, a dropout of 0.06, a weight_decay of 0.0004, and a batch_size of 128. This results in a validation accuracy of 0.64.

D. *random forest*

The achievement of a higher accuracy on random forest classifier requires training models with three hundreds trees and 15,000 left nodes per tree decision in the forest.

As we run through the different number of depths, training accuracy increases as we go deeper trees.

V. Experimental Results

TABLE II
WEIGHTED METRICS ON TESTING DATA

Classifier	Accuracy	Precision	Recall	F1
KNN	0.517692	0.499113	0.517692	0.503445
Logistic				
NN	0.619231	0.6178	0.619231	0.615638
RF	0.653077	0.673374	0.653077	0.635725
Ensemble	0.651538	0.658542	0.651538	0.641197

As can be seen from the table, the best model for this dataset was random forest. This is amazing, since random forest is fast, interpretable, and simple to implement. It is also surprising, since random forest tends to overfit. What's even more surprising is that ensemble with `sklearn.StackingClassifier` and `cv=StratifiedKFold` performs worse than random forest. This makes sense since random forest is such a strong model, adding more models will only make it prone to overfitting, and the `cv=StratifiedKFold` necessary to make training possible with such a large class imbalance also will cause the model to generalize to a more even class distribution which will hurt the testing accuracy on this data. We can see this balance in the statistics, since the ensemble has higher f1 score, and all the scores are much closer, whereas random forest has high precision and lower recall, creating an even lower f1 score.

VI. Insights

We were surprised at how low our testing accuracy turned out to be. On many other datasets, $< 70\%$ would be considered an abysmal result. However, researchers in ML have consistently struggled in getting above 70% in wine [1], [2]. Our results are close to what the limits of the noise from the data would permit. In short, wine tasters are really inconsistent, so creating a model that can model what they, say is impossible beyond a certain accuracy.

There are many methods papers use to address this that we have not address, for example binning the data into 3 or 2 classes instead of 0-10 quality indicators [3]. This helps limit the noise by grouping similar wine qualities. As a result of the subjective measure of human rating, it is likely that the wine is not consistently enough to get a higher rating. We decided not to do this because the resulting classifier would not be able to give precise quality estimates in a production environment, and is therefore less useful.

Other works will also train a separate predictor for red and white wine, whereas we make it a feature [1]. We decided not to do this, because with only 1599 red wines, the red wine predictor would have far too little training data. We determined from the initial analysis that red and white wines generally had similar trends, so a model can transfer learning from one to the other. The decision to have a single model

also simplifies our graphs, sped up development, and reduced deployment complexity.

We were Surprised that random forest did so well. Typically on small datasets, one would expect random forest to overfit, however something about the data seems to lend itself to perform particularly well in random forest. We were even more surprised doing further research and finding that random forest is the agreed upon approach to get higher accuracy [1], [2].

It seems like KNN was the most prone to overfitting, and has one of the lower accuracies. random forest was less prone to overfitting, especially when reducing the tree size, and had the higher accuracy. When we attempted to stratify the data, KNN and random forest were the ones that failed most spectacularly, often learning the testing data from data being duplicated and split between training and testing. Neural Network was less prone to overfitting, however it is the slowest to train and infer, has the most complexity, has the least existing research on it's performance on wine, and had far more hyperparameters to tune in order to get a decent score. Finally, logistic was able to quickly train a solid model that is extremely robust.

In real world data, the performance of the model depends on the distribution of the data. As a result of not stratifying the data, our models are biased towards the class distribution in the training data. Therefore, if the class distribution of data in the real world follows the class distribution of the training data, then we expect the models to do quite well. However, our models are unable to predict the labels 0-2 and 10, and rarely predict labels outside the range 5-7. Therefore, I would be confident in using random forest on real data with about a 65% accuracy, which is close to the state of the art of 70%, however I don't belive that it would fair well on grading wines that are not of average quality. Further, I think the impact of grading a wine of high quality as average quality would prove significant, even if the model is correct on average, so I would never use this model to, for example, grade wines coming out of a factory. We could improve this aspect using stratification, but that would introduce a lot of wine falsely categorized as high quality, which is arguably more problematic. So overall, it could be used in the real world, under very specific circumstances. If used for actual wine categorization, we would need much cleaner data, and much more data across the classes.

VII. Contributions

Alistair Keiller: Worked on the Neural Network, Ensemble, Data Description, Experimental Results, and Insights.

Cindy: Worked on training and generating figures for the kNN classifier.

VIII. Github

https://github.com/AlistairKeiller/cs178Final

References

[1] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," *Decision support systems*, vol. 47, no. 4, pp. 547–553, 2009.

[2] Y. Er and A. Atasoy, "The classification of white wine and red wine according to their physicochemical qualities," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 4, no. Special Issue-1, pp. 23–26, 2016.

[3] N. Kaur, G. Kaur, P. Aruchamy, and N. Chaudhary, "An integrated approach based on fuzzy logic and machine learning techniques for reliable wine quality prediction," *Procedia Computer Science*, vol. 252, pp. 613–622, 2025.